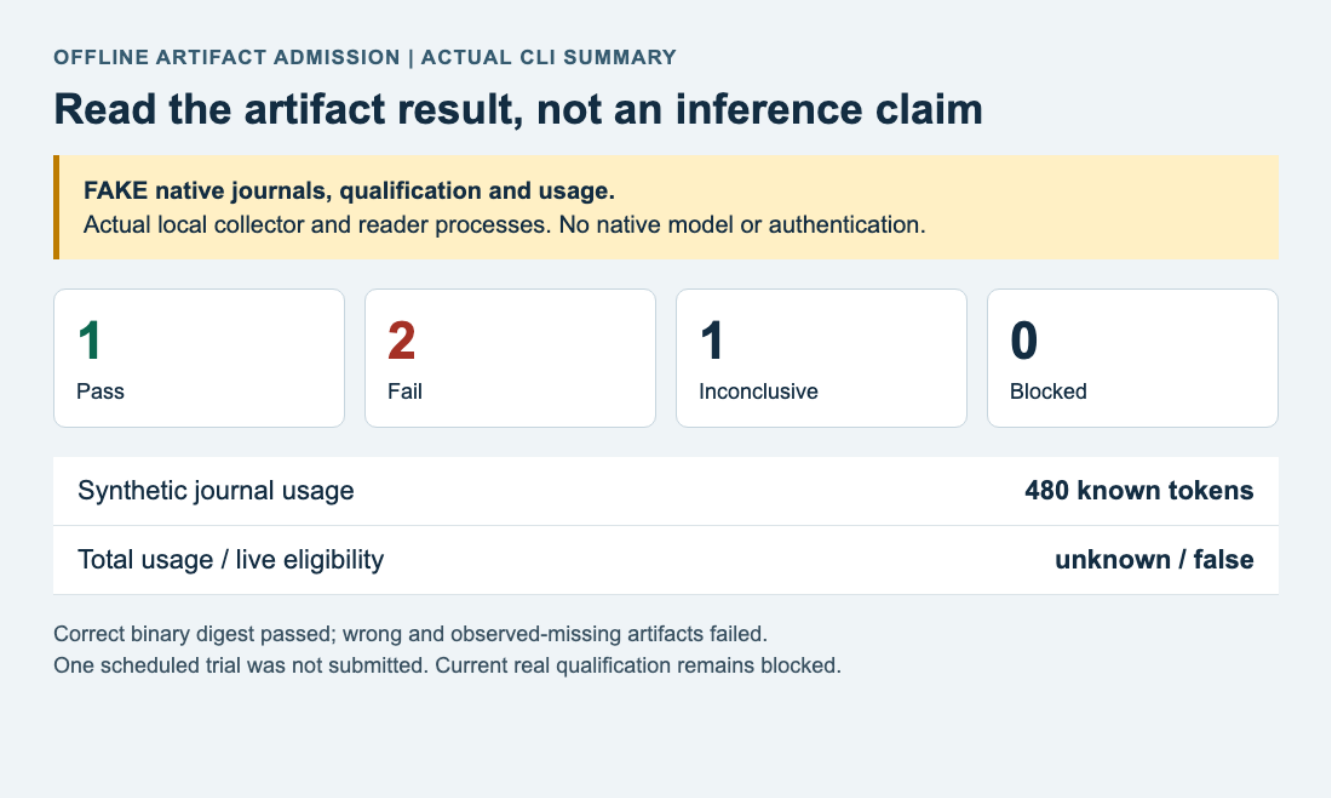
Offline native artifact admission

Evaluation-accounting workflow. Use Python 3.11+ and private frozen inputs. Screenshots are browser-rendered actual CLI summaries with fake native journals, qualification and usage; not terminal images or real inference.

1. Submit recorded artifact evidence

Prepare the private input according to the evaluation-accounting guide. Run trial_admission.py report with JSON stdin and retain the report. Inspect every scheduled trial; exit 0 means a report was produced, not that every trial passed.



Actual collector and reader CLI smoke: synthetic lifecycle evidence. Runtime source bytes match checkpoint fa2ae1202398; execution preceded the commit and was matched afterwards.

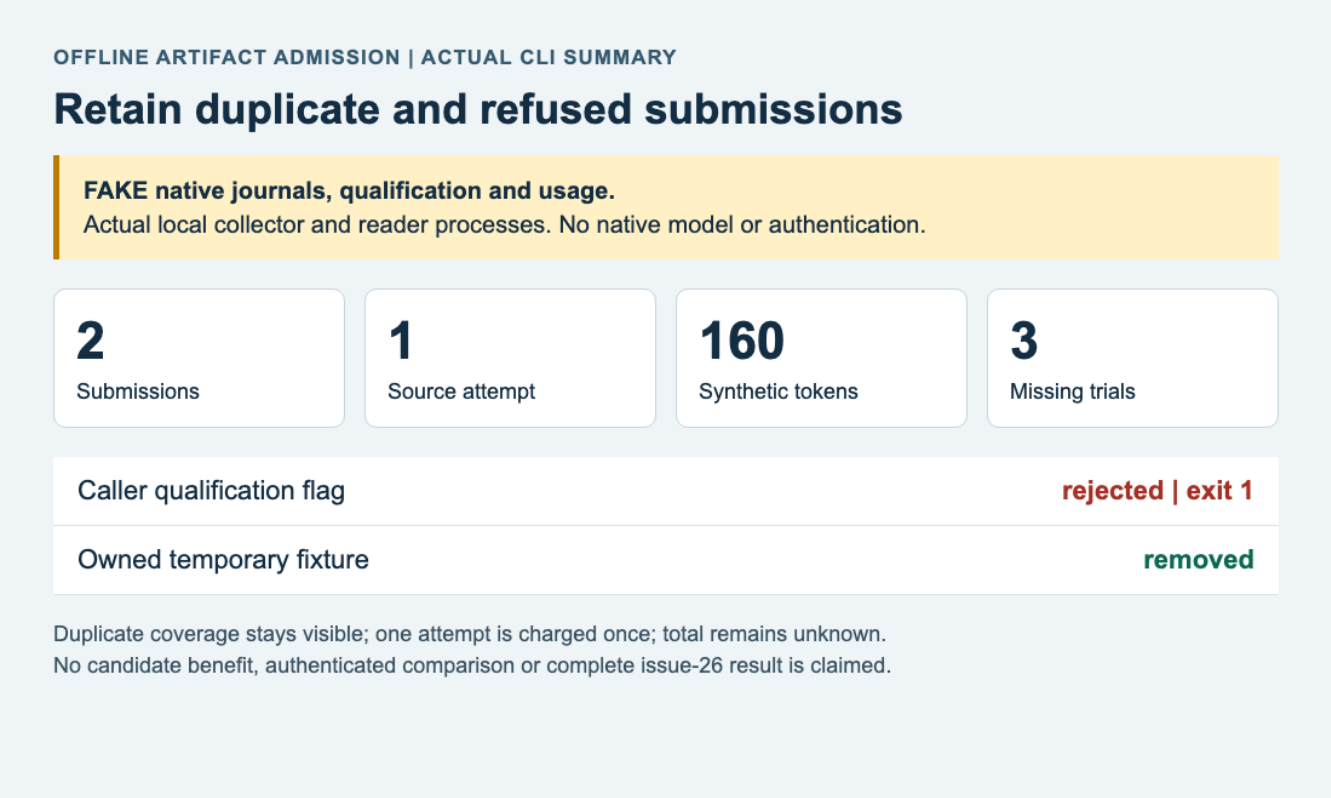
Expected result: One pass, two fails and one unsubmitted inconclusive. Synthetic usage is 480 known tokens; total unknown and live_eligible false. Current real qualification remains blocked.

Offline native artifact admission

Evaluation-accounting workflow. Use Python 3.11+ and private frozen inputs. Screenshots are browser-rendered actual CLI summaries with fake native journals, qualification and usage; not terminal images or real inference.

2. Keep duplicates and refusals visible

Retain failed and duplicate submissions. In an owned synthetic fixture, submit the same attempt twice and verify one cumulative usage charge. Check that a caller qualification flag is rejected and only the owned disposable fixture is removed.



Actual separate CLI results from fake journals: two submissions, one source attempt; arbitrary qualification flag rejected. This is no authentication or model proof.

Expected result: Duplicate coverage and missing trials remain visible. Synthetic 160 known tokens are a subtotal; total unknown. Explicit model export and launch reservations remain separate requirements.